

ML_F16_FFI Community Edition Guide

F-16C Fuel Flow Indicator for DCS World

Build: F16_FUEL_FLOW_COMMUNITY

Phase: COMMUNITY_EDITION_V1

Target hardware: Arduino Nano + SH1106 128×64 I2C OLED

Display library: U8g2 page-buffer mode

DCS interface: DCS-BIOS FP / F-16C fork

Panel: F-16C Fuel Flow Indicator / FFI

1. Purpose of the Community Edition

The Community Edition is a simplified, working version of the F-16C Fuel Flow Indicator intended for cockpit builders who want to drive a small OLED display from DCS-BIOS.

It provides a functional five-digit fuel-flow display that behaves like the aircraft-style counter display:

[10K] [1K] [100] [0] [0]

The first three digits are driven from DCS-BIOS fuel-flow counter values. The final two digits are fixed zeros, giving a fuel-flow display in pounds per hour.

Example:

02300

represents approximately **2,300 PPH**.

The Community Edition is designed to be understandable, modifiable, and suitable for public release while keeping the more advanced commercial/prototype logic out of the public codebase.

2. What This Version Includes

The Community Edition includes:

Feature	Included
DCS-BIOS live fuel-flow input	Yes
Arduino Nano support	Yes
SH1106 128×64 OLED support	Yes
U8g2 page-buffer rendering	Yes
5-digit cockpit-style fuel-flow display	Yes
Startup splash	Yes
Basic digit self-test	Yes
Standby display	Yes
Sync state	Yes
Live display	Yes
Stale-data display	Yes
Rear diagnostics button	Yes
Simple diagnostics page	Yes
Local test mode	Yes
Public calibration table	Yes

3. What Has Been Deliberately Removed

The Community Edition deliberately excludes the more advanced project and product features.

Removed or simplified items include:

Area	Community Edition Position
Commercial / Pro edition switches	Removed
Advanced maintenance mode	Removed
Serial capture logging	Removed
Extended service diagnostics	Removed
Advanced failure/test routines	Removed
Project-specific branding	Removed
Commercial support tooling	Removed

The aim is to provide the community with a useful working gauge, without publishing the more advanced diagnostic, productisation, or service features intended for later protected versions.

4. Hardware Required

4.1 Core Hardware

Recommended hardware:

Item	Notes
Arduino Nano	Standard ATmega328P Nano or compatible
SH1106 128×64 I2C OLED	1.3 inch OLED recommended
Rear diagnostics button	Momentary pushbutton, normally open
USB cable	For programming and DCS-BIOS serial connection
5V supply	Normally via USB from the DCS-BIOS host PC

The sketch is written for an SH1106 128×64 display using hardware I2C.

The display object is:

```
U8G2_SH1106_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0);
```

If your OLED uses a different controller, you may need to change this constructor.

5. Basic Wiring

5.1 OLED Wiring

Typical Arduino Nano I2C wiring:

OLED Pin	Arduino Nano Pin
VCC	5V or 3.3V, depending on display module
GND	GND
SDA	A4
SCL	A5

Most common SH1106 I2C OLED modules work from 5V logic when used with an Arduino Nano, but check your module before wiring.

5.2 Rear Diagnostics Button

The Community Edition uses a rear diagnostics button on pin 7.

```
#define PIN_DIAG_BUTTON 7
```

The button should be wired between:

Button Side	Connection
One side	Arduino Nano D7
Other side	GND

The sketch uses the Arduino internal pull-up resistor:

```
pinMode(PIN_DIAG_BUTTON, INPUT_PULLUP);
```

So the button reads:

State	Reading
Not pressed	HIGH
Pressed	LOW

6. Required Arduino Libraries

Install the following Arduino libraries:

Library	Purpose
U8g2	OLED rendering
Wire	I2C support, normally included with Arduino IDE
DCS-BIOS	DCS serial interface

The sketch uses:

```
#include <Wire.h>
#include <U8g2lib.h>
#define DCSBIOS_IRQ_SERIAL
#include "DcsBios.h"
```

7. DCS-BIOS Requirement

This sketch is intended to be used with the **Skunk Works DCS-BIOS fork** for the DCS F-16C.

The standard/older DCS-BIOS Hub builds may not expose all of the required F-16C fuel-flow counter drum outputs.

For this Community Edition, users should install the Skunk Works fork because it provides the F-16C-specific addresses used by the sketch.

This sketch expects the F-16C DCS-BIOS fork/channel set that exposes the fuel-flow counter drums.

The sketch uses these three DCS-BIOS inputs:

F_16C_50_FUELFLOWCOUNTER_10K
F_16C_50_FUELFLOWCOUNTER_1K
F_16C_50_FUELFLOWCOUNTER_100

These represent:

DCS-BIOS Channel	Display Digit
F_16C_50_FUELFLOWCOUNTER_10K	Ten-thousands digit
F_16C_50_FUELFLOWCOUNTER_1K	Thousands digit
F_16C_50_FUELFLOWCOUNTER_100	Hundreds digit

The final two display digits are fixed zeros.

This means the OLED displays fuel flow in this form:

10K 1K 100 0 0

For example:

Display	Meaning
00800	800 PPH
02300	2,300 PPH
07400	7,400 PPH
19600	19,600 PPH

8. DCS-BIOS Serial Mode

The Community Edition uses:

```
#define DCSBIOS_IRQ_SERIAL
```

This is suitable for an Arduino Nano connected directly over USB serial.

In normal use:

1. Upload the sketch to the Arduino Nano.
 2. Connect the Nano to the DCS-BIOS host PC by USB.
 3. Configure DCS-BIOS / DCS-BIOS Bridge to communicate with the Nano serial port.
 4. Start DCS World.
 5. Load the F-16C.
 6. Once DCS-BIOS begins sending fuel-flow data, the display moves from standby into sync/live operation.
-

9. Main Configuration Switches

The sketch contains a small set of community-safe compile-time switches.

```
#define USE_DCS_BIOS          1
#define USE_TEST_VALUES      0
#define ENABLE_DIAGNOSTICS   1
#define ENABLE_REAR_DIAG_BTN 1
#define SHOW_STARTUP_SPLASH 1
#define ENABLE_SELF_TEST     1
#define START_IN_DIAGS       0
```

9.1 Normal Cockpit Use

Recommended normal configuration:

```
#define USE_DCS_BIOS          1
#define USE_TEST_VALUES      0
#define ENABLE_DIAGNOSTICS   1
#define ENABLE_REAR_DIAG_BTN 1
#define SHOW_STARTUP_SPLASH 1
#define ENABLE_SELF_TEST     1
#define START_IN_DIAGS       0
```

9.2 Bench Test Without DCS

For testing the OLED without DCS-BIOS live data:

```
#define USE_DCS_BIOS          0
#define USE_TEST_VALUES      1
```

This allows the display to count through simulated values.

9.3 Start Directly in Diagnostics

For diagnostics testing:

```
#define START_IN_DIAGS       1
```

Return this to 0 for normal cockpit use.

10. Display Behaviour

10.1 Startup Splash

On power-up, the display shows a simple splash screen:

```
F-16C FFI  
FUEL FLOW  
COMMUNITY EDITION  
DCS-BIOS
```

This confirms that the display has initialised correctly.

10.2 Self-Test

If enabled, the digit self-test cycles all display positions through 0–9.

This is useful for checking:

- OLED operation
- Font rendering
- Digit alignment
- Display visibility
- Basic Arduino timing

Disable with:

```
#define ENABLE_SELF_TEST 0
```

10.3 Standby Mode

Before DCS-BIOS data is received, the display enters standby.

The standby display flashes zeros:

```
00000
```

This means the gauge is powered, but no live DCS fuel-flow data has been seen yet.

10.4 Sync Mode

When DCS-BIOS data is first detected, the gauge briefly enters sync mode.

This confirms that DCS data has started arriving and the display is transitioning from standby to live.

10.5 Live Mode

In live mode, the display shows fuel flow from DCS-BIOS.

Example:

03200

This represents approximately **3,200 PPH**.

10.6 Stale Mode

If DCS-BIOS data stops updating for more than the configured timeout, the gauge enters stale mode.

```
#define DCS_TIMEOUT_MS 3000
```

In stale mode, the last known value remains visible but is drawn inverted.

This gives a clear visual indication that the value is no longer being refreshed.

Possible causes:

- DCS paused
 - Aircraft unloaded
 - DCS-BIOS stopped
 - Serial port disconnected
 - DCS-BIOS Bridge not connected
 - USB cable issue
-

11. System State Model

The Community Edition keeps the standard panel state model, but with simplified behaviour.

```
enum SystemState {  
    SYS_OFF,  
    SYS_BOOT,  
    SYS_STANDBY,  
    SYS_SYNC,  
    SYS_LIVE,  
    SYS_STALE,  
    SYS_DIAG  
};
```

State	Meaning
SYS_OFF	Blank/inactive display state
SYS_BOOT	Startup splash/self-test phase
SYS_STANDBY	Powered but no DCS-BIOS data yet
SYS_SYNC	First DCS data detected, short transition state
SYS_LIVE	Normal live fuel-flow display
SYS_STALE	DCS data has stopped updating
SYS_DIAG	Rear-button diagnostics page

This gives the Community Edition the same broad lifecycle behaviour as the wider ML_F16 panel family, but without the protected advanced diagnostics or maintenance features.

12. Fuel Flow Drum Calibration Table

The sketch includes a public calibration table:

```
const DrumCalPoint fuelFlowDrumTable[] = {  
  {0, 0},  
  {1, 6553},  
  {2, 13106},  
  {3, 19659},  
  {4, 26212},  
  {5, 32765},  
  {6, 39318},  
  {7, 45871},  
  {8, 52424},  
  {9, 58977}  
};
```

The table maps raw DCS-BIOS drum values to displayed digits.

The DCS-BIOS drum outputs are not treated as simple numeric digits during transitions. The table allows the sketch to decode the nearest stable digit and provide basic rolling movement between digits.

12.1 Why the Table Is Needed

A simple conversion such as:

```
digit = raw / 6553;
```

is not always good enough for a drum-style display because the raw value may sit between two digits while the simulated drum is rolling.

The table-based method allows the sketch to:

- Find the nearest digit
- Detect whether the value is close enough to settle
- Work out the next digit
- Apply a simple vertical offset for rolling motion

12.2 Adjusting the Table

If your DCS-BIOS fork gives slightly different raw values, adjust the raw values in the table.

Recommended method:

1. Enable diagnostics.
2. Run the aircraft at known fuel-flow values.
3. Observe the raw drum values for each digit.

4. Update the table if required.
5. Re-upload and test again.

For most users using the same F-16C DCS-BIOS fork, the supplied table should be a workable starting point.

13. Mechanical Drum Rendering

The display uses a simple mechanical-drum visual effect.

Each live DCS digit has:

```
uint8_t digit;  
uint8_t nextDigit;  
int8_t offsetY;
```

These fields are used to draw:

- The current digit
- The next digit
- A vertical offset while transitioning

The function responsible for the visual drum effect is:

```
void drawMechanicalDrum(int x, int y, uint8_t digit, uint8_t nextDigit,  
int8_t offsetY)
```

The final two digits are fixed zeros using:

```
void drawFixedDigit(int x, int y, uint8_t digit)
```

This keeps the display visually close to a mechanical fuel-flow counter without exposing the more advanced drum-rendering logic from the protected branch.

14. Diagnostics Page

The Community Edition includes a simple diagnostics page.

Enter diagnostics by holding the rear diagnostics button for approximately 1.2 seconds:

```
#define DIAG_HOLD_MS 1200
```

The diagnostics page shows:

Field	Meaning
FF	Rendered fuel-flow value
S	Current system state
10K	Raw/digit/offset for 10K drum
1K	Raw/digit/offset for 1K drum
100	Raw/digit/offset for 100 drum
DCS	Whether DCS data has been seen

Example format:

```
FF:02300    S:DIAG
10K R0      D0 00
1K  R13106 D2 00
100 R19659 D3 00
DCS:SEEN
```

To exit diagnostics, hold the rear diagnostics button again.

15. Local Test Mode

Local test mode allows the OLED and digit rendering to be tested without DCS World running.

Enable it with:

```
#define USE_DCS_BIOS      0
#define USE_TEST_VALUES  1
```

The sketch will simulate fuel-flow changes and drive the display locally.

Use this mode to check:

- OLED wiring
- Font size
- Digit placement
- Window mask
- Drum rendering
- Diagnostics button
- Enclosure visibility

Return to normal DCS mode afterwards:

```
#define USE_DCS_BIOS      1
#define USE_TEST_VALUES  0
```

16. Upload Procedure

1. Open the Community Edition .ino file in the Arduino IDE.
 2. Select the correct board, usually **Arduino Nano**.
 3. Select the correct processor option if required.
 - Many clone Nanos require **ATmega328P Old Bootloader**.
 4. Select the correct COM port.
 5. Install the required libraries.
 6. Compile the sketch.
 7. Upload to the Nano.
 8. Open DCS-BIOS / DCS-BIOS Bridge.
 9. Confirm the Nano serial port is connected.
 10. Start DCS World and load the F-16C.
 11. Confirm the display moves from standby to live.
-

17. Recommended Arduino IDE Settings

Typical Arduino Nano settings:

Setting	Value
Board	Arduino Nano
Processor	ATmega328P or ATmega328P Old Bootloader
Port	Your Nano COM port
Programmer	AVRISP mkII, default is usually fine

If upload fails, try switching between:

- ATmega328P
 - ATmega328P (Old Bootloader)
-

18. DCS-BIOS / DCS-BIOS Bridge Notes

The recommended connection method for this Community Edition is:

DCS World → Skunk Works DCS-BIOS Fork → DCS-BIOS Bridge → Arduino Nano over USB serial

Users should use **DCS-BIOS Bridge** to connect the Arduino Nano COM port to DCS-BIOS. This avoids relying on older direct-serial workflows and makes it easier to select, reconnect, and troubleshoot the Nano serial connection.

For DCS-BIOS operation:

1. Install the **Skunk Works DCS-BIOS fork** into the DCS Saved Games folder.
2. Confirm the F-16C export definitions include the required fuel-flow counter channels.
3. Start **DCS-BIOS Bridge**.
4. Select the Arduino Nano COM port in DCS-BIOS Bridge.
5. Confirm the serial connection is active.
6. Do not leave the Arduino Serial Monitor open while DCS-BIOS Bridge is using the port.
7. Start DCS World and load a mission with the F-16C.
8. Confirm the FFI display changes from standby to live once fuel-flow data is received.

If the display stays in standby, the Arduino is running but no valid DCS-BIOS data is being received.

In that case, check:

- The Skunk Works DCS-BIOS fork is installed, not an older unsupported DCS-BIOS package.
 - DCS-BIOS Bridge is running.
 - The correct Nano COM port is selected in DCS-BIOS Bridge.
 - The serial port is not open in Arduino Serial Monitor.
 - The F-16C is loaded in DCS.
 - The required F_16C_50_FUELFLOWCOUNTER_* channels exist in the installed DCS-BIOS fork.
-

19. Troubleshooting

19.1 OLED Blank

Check:

- OLED VCC and GND
- SDA to A4
- SCL to A5
- Correct display controller constructor
- Correct I2C address/display variant
- U8g2 library installed

Try an example U8g2 SH1106 sketch first.

19.2 Upload Fails

Check:

- Correct COM port
- Correct Nano processor setting
- USB cable supports data, not just charging
- No other program is using the serial port
- Try Old Bootloader option

19.3 Display Shows Splash But Never Goes Live

Likely DCS-BIOS issue.

Check:

- DCS-BIOS installed correctly
- DCS-BIOS Bridge running
- Correct serial port selected
- DCS mission loaded
- F-16C aircraft active
- Required F-16C fork/channel names available

19.4 Display Goes Live Then Inverts

The display has entered stale mode.

Likely causes:

- DCS paused

- DCS-BIOS stopped sending data
- Serial connection interrupted
- Aircraft changed/unloaded
- USB cable or hub issue

19.5 Digits Are Misaligned

Adjust:

```
#define DIGIT_SPACING      24
#define DIGIT_BASE_Y      52
#define DIGIT_SCROLL_TRAVEL 40
```

For a different OLED, font, or enclosure aperture, these may need minor tuning.

19.6 Diagnostics Button Does Not Work

Check:

- Button wired between D7 and GND
 - ENABLE_DIAGNOSTICS is set to 1
 - ENABLE_REAR_DIAG_BTN is set to 1
 - Button is normally open
 - Button is held long enough
-

20. Code Structure Overview

The sketch is divided into clear sections:

Section	Purpose
DCS-BIOS config	Selects DCS serial mode
Display libraries	Loads Wire and U8g2
OLED config	Defines SH1106 display object
Build info	Identifies the community build
Feature switches	Enables/disables community features
Pin config	Rear diagnostics button pin
Timing	Update rate, timeout, splash timing
Display config	Digit layout and mask positions
Calibration table	Raw-to-digit mapping
System states	Boot, standby, sync, live, stale, diag
Drum channel structure	Stores raw/digit/offset values
DCS callbacks	Receives DCS-BIOS values
State machine	Controls display mode
Renderers	Draws boot, standby, live, stale, diagnostics
Setup/loop	Arduino lifecycle

This structure is intentionally simple so that other builders can understand and modify it.

21. Safe Areas for Community Modification

Community builders can safely modify:

Area	Example
OLED constructor	For SSD1306 or different SH1106 module
Digit spacing	To suit a different aperture
Splash text	To personalise the startup screen
Diagnostics pin	If D7 is used elsewhere
Timeout value	If stale detection is too sensitive
Calibration table	If their DCS-BIOS fork differs
Test mode speed	To slow down/speed up bench test

Recommended configurable items:

```
#define PIN_DIAG_BUTTON      7
#define UPDATE_INTERVAL_MS  50
#define DCS_TIMEOUT_MS      3000
#define DIGIT_SPACING        24
#define DIGIT_BASE_Y         52
#define DIGIT_SCROLL_TRAVEL  40
```

22. Areas Not Included in the Community Edition

This public version does not include:

- Product build-control framework
- Advanced diagnostics pages
- Service-mode workflow
- NVG/dim mode integration
- DCS lighting proxy integration
- FLCS/test-panel integration
- Extended serial raw capture
- Advanced drum realism logic
- Commercial support routines
- Protected project branding

These omissions are intentional.

The Community Edition should be treated as a practical public release and learning build, not the full internal/professional branch.

23. Community Release Notes

Release text:

This is a community edition F-16C Fuel Flow Indicator sketch for DCS World.

It is designed for an Arduino Nano and SH1106 128x64 I2C OLED display using U8g2 and DCS-BIOS. The display shows the F-16C fuel-flow counter as a five-digit cockpit-style readout using the DCS-BIOS F-16C fuel-flow counter drum outputs.

This release includes startup splash, digit self-test, standby/live/stale states, a simple rear-button diagnostics page, local test mode, and a public calibration table.

Advanced commercial/prototype features have deliberately been removed from this community edition.

24. Disclaimer

Disclaimer for public release:

This project is a hobby cockpit-simulator component for use with DCS World. It is not affiliated with, endorsed by, or approved by Eagle Dynamics, Lockheed Martin, the United States Air Force, or any aircraft manufacturer/operator. It is intended for simulation and educational use only.

25. Licence Position

Licence Type	Effect
Custom hobby-use licence	Allows hobby use but restricts commercial resale

For this project, a sensible public approach may be:

- Code: Custom non-commercial hobby licence
 - Documentation: Creative Commons Attribution-NonCommercial
 - STL/CAD files: separate licence if released
 - Advanced/pro version: kept private
-

26. Quick Start Summary

1. Build the circuit: Arduino Nano + SH1106 OLED + optional rear button.
 2. Install U8g2 and DCS-BIOS libraries.
 3. Open the Community Edition sketch.
 4. Confirm the feature switches are set for normal DCS mode.
 5. Upload to the Nano.
 6. Connect DCS-BIOS / DCS-BIOS Bridge to the Nano COM port.
 7. Start DCS World and load the F-16C.
 8. Confirm the display changes from flashing 00000 to live fuel-flow values.
 9. Hold the rear button to enter diagnostics if required.
 10. Adjust digit positioning or calibration table only if needed.
-

27. Final Notes

The Community Edition is intentionally straightforward. It gives builders a real, working F-16C Fuel Flow Indicator while keeping the more advanced ML_F16 framework, commercial diagnostics, service logic, and protected features out of the public release.

It is a good candidate for a public GitHub release, forum post, or build-log package alongside:

- Wiring diagram
- OLED mounting photo
- Example enclosure photos
- DCS-BIOS setup notes
- Short demo video
- Known-good Arduino IDE/library versions

28. Appendices

28.1 Appendix A – Code

```
/******  
* ML_F16_FUEL_FLOW_INDICATOR  
* PHASE_1E_NANO_OLED_MECH_DRUM_REFINED_TEST_PANEL  
*  
* F-16C Fuel Flow Indicator  
* Arduino Nano + SH1106 128x64 I2C OLED  
* U8g2 Page Buffer + DCS-BIOS FP Fork  
*  
* Features:  
* - Refined nearest-step FFI drum decoder  
* - Top-end rollover handling  
* - Mechanical drum aperture/window mask  
* - Clean cockpit display  
* - STALE inverted display  
* - STANDBY flashing 00000  
* - Rear-button diagnostics fallback  
* - HUD_BRT_SW NIGHT driven NVG/dim proxy  
* - FLCS_PWR_TEST_SW driven TEST / MAINT modes  
*  
* Test panel behaviour:  
* FLCS PWR TEST switch NORM = normal operation  
* FLCS PWR TEST switch TEST = SYS_TEST display self-test  
* FLCS PWR TEST switch MAINT = SYS_DIAG diagnostics/capture page  
*  
* Display layout:  
* [10K] [1K] [100] [0] [0]  
*****/  
  
// -----  
// DCS-BIOS CONFIG  
// -----  
  
#define DCSBIOS_IRQ_SERIAL  
#include "DcsBios.h"  
  
// -----  
// DISPLAY LIBRARIES  
// -----  
  
#include <Wire.h>  
#include <U8g2lib.h>  
  
// -----  
// OLED CONFIG - RAM OPTIMISED  
// -----
```

```

U8G2_SH1106_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0);

// -----
// BUILD INFO
// -----

#define BUILD_NAME      "ML_F16_FUEL_FLOW_INDICATOR"
#define BUILD_PHASE      "PHASE_1E_NANO_OLED_MECH_DRUM_REFINED_TEST_PANEL"
#define BUILD_DATE       "2026-05-22"
#define PANEL_ID         "F16_FFI"
#define HARDWARE_TARGET  "ARDUINO_NANO_SH1106_128x64"

// -----
// EDITION CONTROL
// -----

#define EDITION_COMMUNITY 1
#define EDITION_PRO       0

#if EDITION_PRO
    #define BUILD_EDITION "PRO"
#else
    #define BUILD_EDITION "COMMUNITY"
#endif

// -----
// FRAMEWORK SWITCHES
// -----

#define USE_DCS_BIOS      1
#define USE_TEST_VALUES   0
#define ENABLE_DIAGNOSTICS 1
#define ENABLE_REAR_DIAG_BTN 1
#define SHOW_STARTUP_SPLASH 1
#define ENABLE_SELF_TEST  1

// Normal cockpit boot.
// Rear button and DCS MAINT switch still open diagnostics.
#define START_IN_DIAGS    0

// -----
// DCS TEST PANEL / MAINT MODE CONTROLS
// -----

#define ENABLE_DCS_TEST_PANEL_MODE 1

// DCS-BIOS source:
// FLCS_PWR_TEST_SW = FLCS PWR TEST Switch, MAINT / NORM / TEST
//
// Expected value mapping for this fork:
// 0 = MAINT
// 1 = NORM

```

```

// 2 = TEST
//
// If your test shows TEST/MAINT reversed, swap MAINT and TEST below.

#define FLCS_PWR_TEST_SW_MAINT_VALUE 0
#define FLCS_PWR_TEST_SW_NORM_VALUE 1
#define FLCS_PWR_TEST_SW_TEST_VALUE 2

// -----
// NVG / DIM MODE CONTROLS
// -----

#define ENABLE_NVG_MODE 1
#define ENABLE_DCS_NVG_PROXY 1
#define NVG_PROXY_HUD_NIGHT 1

#define OLED_CONTRAST_NORMAL 255
#define OLED_CONTRAST_NVG 24

// HUD Brightness Control Switch expected values.
// If testing proves NIGHT is not 2 on your fork, change this value only.
#define HUD_BRT_SW_DAY_VALUE 0
#define HUD_BRT_SW_AUTO_VALUE 1
#define HUD_BRT_SW_NIGHT_VALUE 2

// -----
// OPTIONAL SERIAL CAPTURE LOGGING
// -----
//
// Keep off by default. DCS-BIOS IRQ serial should own Serial.

#define ENABLE_SERIAL_CAPTURE_LOG 0

// -----
// PIN CONFIG
// -----

#define PIN_DIAG_BUTTON 7

// -----
// TIMING
// -----

#define UPDATE_INTERVAL_MS 50
#define DCS_TIMEOUT_MS 3000
#define SPLASH_TIME_MS 1200
#define SELF_TEST_DELAY_MS 80
#define DIAG_HOLD_MS 1200
#define STANDBY_BLINK_MS 600
#define TEST_SWEEP_MS 150

// -----
// DIGIT / DRUM DISPLAY CONFIG

```

```

// -----

#define DIGIT_COUNT      5
#define LIVE_DCS_DRUM_COUNT  3

#define DIGIT_SPACING     24
#define DIGIT_BASE_Y      52
#define DIGIT_SCROLL_TRAVEL 40

#define WINDOW_TOP_Y      19
#define WINDOW_BOTTOM_Y   61

#define MASK_TOP_X        0
#define MASK_TOP_Y        0
#define MASK_TOP_W        128
#define MASK_TOP_H        WINDOW_TOP_Y

#define MASK_BOTTOM_X      0
#define MASK_BOTTOM_Y      WINDOW_BOTTOM_Y
#define MASK_BOTTOM_W      128
#define MASK_BOTTOM_H      (64 - WINDOW_BOTTOM_Y)

#define INVERT_BOX_X      0
#define INVERT_BOX_Y      WINDOW_TOP_Y
#define INVERT_BOX_W      128
#define INVERT_BOX_H      (WINDOW_BOTTOM_Y - WINDOW_TOP_Y)

// -----
// DECODER CALIBRATION CONSTANTS
// -----

#define DRUM_STEP_RAW      6553
#define DRUM_SETTLE_DEADBAND 220
#define DRUM_TOP_ROLLOVER_RAW 64800

// -----
// SYSTEM STATES
// -----

enum SystemState {
    SYS_OFF,
    SYS_BOOT,
    SYS_STANDBY,
    SYS_SYNC,
    SYS_LIVE,
    SYS_STALE,
    SYS_TEST,
    SYS_DIAG
};

SystemState currentState = SYS_BOOT;

// -----

```

```

// BRIGHTNESS / NVG STATE
// -----

enum BrightnessMode {
    BRIGHT_NORMAL,
    BRIGHT_NVG
};

BrightnessMode brightnessMode = BRIGHT_NORMAL;

uint8_t hudBrTswRaw = HUD_BRT_SW_DAY_VALUE;
bool dcsNvgProxyActive = false;

// -----
// DCS TEST PANEL STATE
// -----

uint8_t flcsPwrTestSwRaw = FLCS_PWR_TEST_SW_NORM_VALUE;
bool dcsTestPanelForcesTest = false;
bool dcsTestPanelForcesDiag = false;

// Local fallback diagnostics from rear button.
bool localDiagnosticsMode = false;

// -----
// DCS RAW DRUM STRUCTURE
// -----

struct DrumChannel {
    uint16_t raw;
    uint8_t digit;
    uint8_t nextDigit;
    int8_t offsetY;
    unsigned long lastUpdateMs;
    uint16_t updateCount;
    bool live;
};

DrumChannel drums[LIVE_DCS_DRUM_COUNT];

// -----
// GLOBAL STATE
// -----

unsigned long lastFrameMs = 0;
unsigned long lastDcsPacketMs = 0;
unsigned long diagButtonStartMs = 0;
unsigned long syncStartMs = 0;
unsigned long lastTestStepMs = 0;

bool dcsEverSeen = false;
bool syncSeen = false;

```

```

uint16_t testCounter = 0;

// -----
// FORWARD DECLARATIONS
// -----

void updateSystemState();
void renderCurrentState();

void drawBootScreen();
void drawStandbyScreen();
void drawSyncScreen();
void drawLiveDisplay(bool stale);
void drawDiagnostics();
void drawTestScreen();

void renderPage(void (*drawFunc)());
void renderLivePage(bool stale);

void initialiseDrums();
void selfTestDigits();

void updateDrumFromDcs(uint8_t index, unsigned int rawValue);
void decodeMechanicalDrum(uint16_t rawValue, uint8_t &digit, uint8_t &nextDigit, int8_t &offsetY);

void drawMechanicalDrum(int x, int y, uint8_t digit, uint8_t nextDigit, int8_t offsetY);
void drawFixedDigit(int x, int y, uint8_t digit);
void drawDisplayDigits();
void drawZeroDisplay();
void applyWindowMask();

void updateRearDiagnosticsButton();
void updateTestValues();

void applyBrightnessMode();
void updateNvgProxyFromHudSwitch(uint8_t switchValue);
void updateTestPanelModeFromDcs(uint8_t switchValue);

uint16_t getRenderedFuelFlow();
const char* systemStateName(SystemState state);
const char* brightnessModeName();

#if ENABLE_SERIAL_CAPTURE_LOG
void serialCaptureLog();
#endif

// -----
// BASIC HELPERS
// -----

void renderPage(void (*drawFunc)()) {
    u8g2.firstPage();
    do {

```

```

    drawFunc();
} while (u8g2.nextPage());
}

void renderLivePage(bool stale) {
    u8g2.firstPage();
    do {
        drawLiveDisplay(stale);
    } while (u8g2.nextPage());
}

void drawCentreText(int y, const char* text) {
    int w = u8g2.getStrWidth(text);
    int x = (128 - w) / 2;
    if (x < 0) x = 0;
    u8g2.drawStr(x, y, text);
}

const char* systemStateName(SystemState state) {
    switch (state) {
        case SYS_OFF:    return "OFF";
        case SYS_BOOT:   return "BOOT";
        case SYS_STANDBY: return "STBY";
        case SYS_SYNC:   return "SYNC";
        case SYS_LIVE:   return "LIVE";
        case SYS_STALE:  return "STALE";
        case SYS_TEST:   return "TEST";
        case SYS_DIAG:   return "DIAG";
        default:         return "UNK";
    }
}

const char* brightnessModeName() {
    switch (brightnessMode) {
        case BRIGHT_NVG: return "NVG";
        case BRIGHT_NORMAL:
            default:       return "NORM";
    }
}

uint16_t getRenderedFuelFlow() {
    return ((uint16_t)drums[0].digit * 10000U) +
        ((uint16_t)drums[1].digit * 1000U) +
        ((uint16_t)drums[2].digit * 100U);
}

// -----
// NVG / BRIGHTNESS CONTROL
// -----

void applyBrightnessMode() {
#ifdef ENABLE_NVG_MODE
    if (brightnessMode == BRIGHT_NVG) {

```

```

    u8g2.setContrast(OLED_CONTRAST_NVG);
} else {
    u8g2.setContrast(OLED_CONTRAST_NORMAL);
}
#else
    u8g2.setContrast(OLED_CONTRAST_NORMAL);
#endif
}

void updateNvgProxyFromHudSwitch(uint8_t switchValue) {
#if ENABLE_NVG_MODE && ENABLE_DCS_NVG_PROXY && NVG_PROXY_HUD_NIGHT

    hudBrtSwRaw = switchValue;

    if (switchValue == HUD_BRT_SW_NIGHT_VALUE) {
        dcsNvgProxyActive = true;
        brightnessMode = BRIGHT_NVG;
    } else {
        dcsNvgProxyActive = false;
        brightnessMode = BRIGHT_NORMAL;
    }

    // Diagnostics stay readable at normal contrast.
    if (!localDiagnosticsMode && !dcsTestPanelForcesDiag) {
        applyBrightnessMode();
    }

#endif
}

// -----
// DCS TEST PANEL MODE CONTROL
// -----

void updateTestPanelModeFromDcs(uint8_t switchValue) {
#if ENABLE_DCS_TEST_PANEL_MODE

    flcsPwrTestSwRaw = switchValue;

    if (switchValue == FLCS_PWR_TEST_SW_MAINT_VALUE) {
        dcsTestPanelForcesDiag = true;
        dcsTestPanelForcesTest = false;

        // MAINT/DIAG readability at normal contrast.
        u8g2.setContrast(OLED_CONTRAST_NORMAL);
        return;
    }

    if (switchValue == FLCS_PWR_TEST_SW_TEST_VALUE) {
        dcsTestPanelForcesTest = true;
        dcsTestPanelForcesDiag = false;

        // TEST uses active display, apply cockpit brightness mode.

```

```

    applyBrightnessMode();
    return;
}

// NORM or unknown = release DCS-forced modes.
dcsTestPanelForcesDiag = false;
dcsTestPanelForcesTest = false;

if (!localDiagnosticsMode) {
    applyBrightnessMode();
}

#endif
}

// -----
// REFINED MECHANICAL DRUM DECODER
// -----

void decodeMechanicalDrum(uint16_t rawValue, uint8_t &digit, uint8_t &nextDigit, int8_t &offsetY) {
    if (rawValue <= 9) {
        digit = (uint8_t)rawValue;
        nextDigit = (digit + 1) % 10;
        offsetY = 0;
        return;
    }

    if (rawValue >= DRUM_TOP_ROLLOVER_RAW) {
        digit = 0;
        nextDigit = 1;
        offsetY = 0;
        return;
    }

    uint8_t nearestDigit = (uint8_t)((rawValue + (DRUM_STEP_RAW / 2)) / DRUM_STEP_RAW);

    if (nearestDigit > 9) {
        nearestDigit = 9;
    }

    uint16_t nearestRaw = (uint16_t)((uint32_t)nearestDigit * DRUM_STEP_RAW);
    int16_t residual = (int16_t)rawValue - (int16_t)nearestRaw;

    if (residual > -DRUM_SETTLE_DEADBAND && residual < DRUM_SETTLE_DEADBAND) {
        digit = nearestDigit;
        nextDigit = (digit + 1) % 10;
        offsetY = 0;
        return;
    }

    if (residual > 0) {
        digit = nearestDigit;
        nextDigit = (digit + 1) % 10;
    }

```

```

offsetY = -(int8_t)(((uint32_t)residual * DIGIT_SCROLL_TRAVEL) / DRUM_STEP_RAW);

if (offsetY < -DIGIT_SCROLL_TRAVEL) {
    offsetY = -DIGIT_SCROLL_TRAVEL;
}

return;
}

uint8_t prevDigit = (nearestDigit == 0) ? 9 : nearestDigit - 1;

digit = prevDigit;
nextDigit = nearestDigit;

uint16_t absResidual = (uint16_t)(-residual);
offsetY = -(int8_t)(DIGIT_SCROLL_TRAVEL - (((uint32_t)absResidual * DIGIT_SCROLL_TRAVEL) /
DRUM_STEP_RAW));

if (offsetY < -DIGIT_SCROLL_TRAVEL) {
    offsetY = -DIGIT_SCROLL_TRAVEL;
}
}

// -----
// DCS CHANNEL UPDATE
// -----

void updateDrumFromDcs(uint8_t index, unsigned int rawValue) {
    if (index >= LIVE_DCS_DRUM_COUNT) {
        return;
    }

    drums[index].raw = (uint16_t)rawValue;

    decodeMechanicalDrum(
        drums[index].raw,
        drums[index].digit,
        drums[index].nextDigit,
        drums[index].offsetY
    );

    drums[index].lastUpdateMs = millis();

    if (drums[index].updateCount < 65535) {
        drums[index].updateCount++;
    }

    drums[index].live = true;

    lastDcsPacketMs = millis();

    if (!dcsEverSeen) {

```

```

    dcsEverSeen = true;
    syncSeen = false;
    syncStartMs = millis();
}
}

// -----
// DCS-BIOS CALLBACKS - F-16C FUEL FLOW COUNTER
// -----

void onFuelFlowCounter10kChange(unsigned int newValue) {
    updateDrumFromDcs(0, newValue);
}

void onFuelFlowCounter1kChange(unsigned int newValue) {
    updateDrumFromDcs(1, newValue);
}

void onFuelFlowCounter100Change(unsigned int newValue) {
    updateDrumFromDcs(2, newValue);
}

// -----
// DCS-BIOS CALLBACKS - HUD / TEST PANEL
// -----

void onHudBrTswChange(unsigned int newValue) {
    updateNvgProxyFromHudSwitch((uint8_t)newValue);
}

void onFlcsPwrTestSwChange(unsigned int newValue) {
    updateTestPanelModeFromDcs((uint8_t)newValue);
}

// -----
// DCS-BIOS BUFFERS
// -----
//
// In this DCS-BIOS FP Fork, these macros already expand to:
// address, mask, shift
//
// Correct format:
// DcsBios::IntegerBuffer buffer(CONTROL_MACRO, callback);

#if USE_DCS_BIOS

DcsBios::IntegerBuffer fuelFlowCounter10kBuffer(
    F_16C_50_FUELFLOWCOUNTER_10K,
    onFuelFlowCounter10kChange
);

DcsBios::IntegerBuffer fuelFlowCounter1kBuffer(
    F_16C_50_FUELFLOWCOUNTER_1K,

```

```

    onFuelFlowCounter1kChange
);

DcsBios::IntegerBuffer fuelFlowCounter100Buffer(
    F_16C_50_FUELFLOWCOUNTER_100,
    onFuelFlowCounter100Change
);

#if ENABLE_NVG_MODE && ENABLE_DCS_NVG_PROXY
DcsBios::IntegerBuffer hudBrTswBuffer(
    F_16C_50_HUD_BRT_SW,
    onHudBrTswChange
);
#endif

#if ENABLE_DCS_TEST_PANEL_MODE
DcsBios::IntegerBuffer flcsPwrTestSwBuffer(
    F_16C_50_FLCS_PWR_TEST_SW,
    onFlcsPwrTestSwChange
);
#endif

#endif

// -----
// STATE MACHINE
// -----

void updateSystemState() {
    if (localDiagnosticsMode || dcsTestPanelForcesDiag) {
        currentState = SYS_DIAG;
        return;
    }

    if (dcsTestPanelForcesTest) {
        currentState = SYS_TEST;
        return;
    }

    if (!dcsEverSeen) {
        currentState = SYS_STANDBY;
        return;
    }

    if ((millis() - lastDcsPacketMs) > DCS_TIMEOUT_MS) {
        currentState = SYS_STALE;
        return;
    }

    if (!syncSeen) {
        currentState = SYS_SYNC;

        if ((millis() - syncStartMs) > 800) {

```

```

        syncSeen = true;
    }

    return;
}

currentState = SYS_LIVE;
}

// -----
// DIAGNOSTICS BUTTON
// -----

void updateRearDiagnosticsButton() {
#ifdef ENABLE_DIAGNOSTICS && ENABLE_REAR_DIAG_BTN

    bool pressed = digitalRead(PIN_DIAG_BUTTON) == LOW;

    if (pressed) {
        if (diagButtonStartMs == 0) {
            diagButtonStartMs = millis();
        }

        if ((millis() - diagButtonStartMs) > DIAG_HOLD_MS) {
            localDiagnosticsMode = !localDiagnosticsMode;
            diagButtonStartMs = 0;

            if (localDiagnosticsMode) {
                u8g2.setContrast(OLED_CONTRAST_NORMAL);
            } else {
                applyBrightnessMode();
            }

            delay(350);
        }
        else {
            diagButtonStartMs = 0;
        }
    }

#endif
}

// -----
// TEST VALUES
// -----

void updateTestValues() {
#ifdef USE_TEST_VALUES

    if ((millis() - lastTestStepMs) < 120) {
        return;
    }

```

```

lastTestStepMs = millis();

testCounter += 100;

if (testCounter > 99900) {
    testCounter = 0;
}

uint8_t d0 = (testCounter / 10000) % 10;
uint8_t d1 = (testCounter / 1000) % 10;
uint8_t d2 = (testCounter / 100) % 10;

drums[0].raw = d0;
drums[1].raw = d1;
drums[2].raw = d2;

for (uint8_t i = 0; i < LIVE_DCS_DRUM_COUNT; i++) {
    decodeMechanicalDrum(
        drums[i].raw,
        drums[i].digit,
        drums[i].nextDigit,
        drums[i].offsetY
    );

    drums[i].live = true;
    drums[i].lastUpdateMs = millis();
}

dcsEverSeen = true;

if (!syncSeen && syncStartMs == 0) {
    syncStartMs = millis();
}

lastDcsPacketMs = millis();

#endif
}

// -----
// INITIALISATION
// -----

void initialiseDrums() {
    for (uint8_t i = 0; i < LIVE_DCS_DRUM_COUNT; i++) {
        drums[i].raw = 0;
        drums[i].digit = 0;
        drums[i].nextDigit = 1;
        drums[i].offsetY = 0;
        drums[i].lastUpdateMs = 0;
        drums[i].updateCount = 0;
        drums[i].live = false;
    }
}

```

```

}

// -----
// SELF TEST
// -----

void selfTestDigits() {
#ifdef ENABLE_SELF_TEST

    for (uint8_t d = 0; d <= 9; d++) {
        u8g2.firstPage();

        do {
            u8g2.setFont(u8g2_font_logisoso32_tn);

            char str[2];
            str[0] = '0' + d;
            str[1] = '\0';

            for (uint8_t i = 0; i < DIGIT_COUNT; i++) {
                int x = i * DIGIT_SPACING;
                u8g2.drawStr(x, DIGIT_BASE_Y, str);
            }

        } while (u8g2.nextPage());

        delay(SELF_TEST_DELAY_MS);
    }

#endif

// -----
// DIGIT DRAWING
// -----

void drawMechanicalDrum(int x, int y, uint8_t digit, uint8_t nextDigit, int8_t offsetY) {
    char buf[2];

    buf[0] = '0' + digit;
    buf[1] = '\0';
    u8g2.drawStr(x, y + offsetY, buf);

    if (offsetY != 0) {
        buf[0] = '0' + nextDigit;
        buf[1] = '\0';
        u8g2.drawStr(x, y + offsetY + DIGIT_SCROLL_TRAVEL, buf);
    }
}

void drawFixedDigit(int x, int y, uint8_t digit) {
    char buf[2];

```

```

    buf[0] = '0' + digit;
    buf[1] = '\0';

    u8g2.drawStr(x, y, buf);
}

void applyWindowMask() {
    u8g2.setDrawColor(0);

    u8g2.drawBox(MASK_TOP_X, MASK_TOP_Y, MASK_TOP_W, MASK_TOP_H);
    u8g2.drawBox(MASK_BOTTOM_X, MASK_BOTTOM_Y, MASK_BOTTOM_W, MASK_BOTTOM_H);

    u8g2.setDrawColor(1);
}

void drawDisplayDigits() {
    u8g2.setFont(u8g2_font_logisoso32_tn);

    for (uint8_t i = 0; i < LIVE_DCS_DRUM_COUNT; i++) {
        int posX = i * DIGIT_SPACING;

        drawMechanicalDrum(
            posX,
            DIGIT_BASE_Y,
            drums[i].digit,
            drums[i].nextDigit,
            drums[i].offsetY
        );
    }

    drawFixedDigit(3 * DIGIT_SPACING, DIGIT_BASE_Y, 0);
    drawFixedDigit(4 * DIGIT_SPACING, DIGIT_BASE_Y, 0);

    applyWindowMask();
}

void drawZeroDisplay() {
    u8g2.setFont(u8g2_font_logisoso32_tn);

    for (uint8_t i = 0; i < DIGIT_COUNT; i++) {
        drawFixedDigit(i * DIGIT_SPACING, DIGIT_BASE_Y, 0);
    }

    applyWindowMask();
}

// -----
// RENDERERS
// -----

void drawBootScreen() {
    u8g2.setFont(u8g2_font_6x10_tf);
    drawCentreText(12, "ML F-16C SIM");
}

```

```

u8g2.setFont(u8g2_font_7x14B_tf);
drawCentreText(30, "FUEL FLOW");

u8g2.setFont(u8g2_font_5x8_tf);
drawCentreText(45, "TEST PANEL");
drawCentreText(60, "PHASE 1E");
}

void drawStandbyScreen() {
    bool blinkOn = ((millis() / STANDBY_BLINK_MS) % 2) == 0;

    if (blinkOn) {
        drawZeroDisplay();
    }
}

void drawSyncScreen() {
    drawDisplayDigits();
}

void drawLiveDisplay(bool stale) {
    if (stale) {
        u8g2.drawBox(INVERT_BOX_X, INVERT_BOX_Y, INVERT_BOX_W, INVERT_BOX_H);
        u8g2.setDrawColor(0);

        u8g2.setFont(u8g2_font_logisoso32_tn);

        for (uint8_t i = 0; i < LIVE_DCS_DRUM_COUNT; i++) {
            int posX = i * DIGIT_SPACING;

            drawMechanicalDrum(
                posX,
                DIGIT_BASE_Y,
                drums[i].digit,
                drums[i].nextDigit,
                drums[i].offsetY
            );
        }

        drawFixedDigit(3 * DIGIT_SPACING, DIGIT_BASE_Y, 0);
        drawFixedDigit(4 * DIGIT_SPACING, DIGIT_BASE_Y, 0);

        u8g2.setDrawColor(1);
        applyWindowMask();
    } else {
        drawDisplayDigits();
    }
}

void drawTestScreen() {
    u8g2.setFont(u8g2_font_logisoso32_tn);

```

```

uint8_t d = (millis() / TEST_SWEEP_MS) % 10;

for (uint8_t i = 0; i < DIGIT_COUNT; i++) {
    drawFixedDigit(i * DIGIT_SPACING, DIGIT_BASE_Y, d);
}

applyWindowMask();
}

void drawDiagnostics() {
    char buf[32];

    u8g2.setFont(u8g2_font_5x8_tf);

    snprintf(buf, sizeof(buf), "ARD:%05u", getRenderedFuelFlow());
    u8g2.drawStr(0, 8, buf);

    snprintf(buf, sizeof(buf), "S:%s %s", systemStateName(currentState), brightnessModeName());
    u8g2.drawStr(64, 8, buf);

    snprintf(buf, sizeof(buf), "10K R%u D%u O%d", drums[0].raw, drums[0].digit, drums[0].offsetY);
    u8g2.drawStr(0, 22, buf);

    snprintf(buf, sizeof(buf), "1K R%u D%u O%d", drums[1].raw, drums[1].digit, drums[1].offsetY);
    u8g2.drawStr(0, 34, buf);

    snprintf(buf, sizeof(buf), "100 R%u D%u O%d", drums[2].raw, drums[2].digit, drums[2].offsetY);
    u8g2.drawStr(0, 46, buf);

    snprintf(buf, sizeof(buf), "HUD:%u FLCS:%u", hudBrtsSwRaw, flcsPwrTestSwRaw);
    u8g2.drawStr(0, 62, buf);
}

void renderCurrentState() {
    switch (currentState) {
        case SYS_BOOT:
            renderPage(drawBootScreen);
            break;

        case SYS_STANDBY:
            renderPage(drawStandbyScreen);
            break;

        case SYS_SYNC:
            renderPage(drawSyncScreen);
            break;

        case SYS_LIVE:
            renderLivePage(false);
            break;

        case SYS_STALE:
            renderLivePage(true);

```

```

        break;

    case SYS_TEST:
        renderPage(drawTestScreen);
        break;

    case SYS_DIAG:
        renderPage(drawDiagnostics);
        break;

    case SYS_OFF:
    default:
        u8g2.firstPage();
        do {
        } while (u8g2.nextPage());
        break;
    }
}

#ifdef ENABLE_SERIAL_CAPTURE_LOG
void serialCaptureLog() {
    Serial.print(millis());
    Serial.print(',');
    Serial.print(systemStateName(currentState));
    Serial.print(',');
    Serial.print(getRenderedFuelFlow());
    Serial.print(',');
    Serial.print(drums[0].raw);
    Serial.print(',');
    Serial.print(drums[0].digit);
    Serial.print(',');
    Serial.print(drums[0].offsetY);
    Serial.print(',');
    Serial.print(drums[1].raw);
    Serial.print(',');
    Serial.print(drums[1].digit);
    Serial.print(',');
    Serial.print(drums[1].offsetY);
    Serial.print(',');
    Serial.print(drums[2].raw);
    Serial.print(',');
    Serial.print(drums[2].digit);
    Serial.print(',');
    Serial.print(drums[2].offsetY);
    Serial.print(',');
    Serial.print(hudBrTswRaw);
    Serial.print(',');
    Serial.print(dcsNvgProxyActive ? 1 : 0);
    Serial.print(',');
    Serial.print(flcsPwrTestSwRaw);
    Serial.print(',');
    Serial.print(dcsTestPanelForcesTest ? 1 : 0);
    Serial.print(',');
}

```

```

    Serial.println(dcsTestPanelForcesDiag ? 1 : 0);
}
#endif

// -----
// SETUP
// -----

void setup() {
#if ENABLE_REAR_DIAG_BTN
    pinMode(PIN_DIAG_BUTTON, INPUT_PULLUP);
#endif

    initialiseDrums();

    u8g2.begin();
    applyBrightnessMode();

#if SHOW_STARTUP_SPLASH
    currentState = SYS_BOOT;
    renderCurrentState();
    delay(SPLASH_TIME_MS);
#endif

#if ENABLE_SELF_TEST
    selfTestDigits();
#endif

#if ENABLE_SERIAL_CAPTURE_LOG
    Serial.begin(115200);
#endif

#if USE_DCS_BIOS
    DcsBios::setup();
#endif

#if START_IN_DIAGS
    localDiagnosticsMode = true;
    currentState = SYS_DIAG;
    u8g2.setContrast(OLED_CONTRAST_NORMAL);
#else
    currentState = SYS_STANDBY;
    applyBrightnessMode();
#endif
}

// -----
// LOOP
// -----

void loop() {
#if USE_DCS_BIOS
    DcsBios::loop();

```

```
#endif
```

```
    updateRearDiagnosticsButton();
```

```
#if USE_TEST_VALUES
```

```
    updateTestValues();
```

```
#endif
```

```
    updateSystemState();
```

```
    unsigned long now = millis();
```

```
    if ((now - lastFrameMs) >= UPDATE_INTERVAL_MS) {  
        renderCurrentState();
```

```
#if ENABLE_SERIAL_CAPTURE_LOG
```

```
    if (localDiagnosticsMode || dcsTestPanelForcesDiag) {  
        serialCaptureLog();
```

```
    }
```

```
#endif
```

```
    lastFrameMs = now;
```

```
}
```

```
}
```